

SMARTBRIDGE

Let's Bridge the Gap

PROJECT DOCUMENTATION REPORT

Credit Card Approval Prediction System Using Machine Learning

*A Machine Learning based Web Application for
Automated Credit Card Approval Decisioning*

Technology Stack: Python · Flask · Scikit-Learn · Pandas · NumPy · Joblib · HTML/CSS/JavaScript · Ngrok

Project Description

Access to credit is one of the foundational pillars of the modern financial system, and the process by which banks and financial institutions decide whether to extend a credit card to an applicant carries significant weight for both the institution and the individual. Traditionally, credit card approval has depended on manual verification by loan officers, who review income statements, credit bureau reports, employment history, and existing liabilities before arriving at a decision. This manual process, while thorough, is inherently slow, inconsistent across reviewers, and difficult to scale as the volume of applications grows. The Credit Card Approval Prediction System Using Machine Learning is designed to modernize this workflow by using a trained machine learning model to automatically predict whether a credit card application should be Approved or Rejected, based on the applicant's financial and demographic profile.

In the banking and financial services sector, the importance of a reliable, data-driven approval mechanism cannot be overstated. Financial institutions handle thousands of applications daily, and even marginal improvements in the speed and consistency of the approval process translate into significant operational savings and improved customer experience. A machine learning based approach analyses historical patterns in applicant data — such as annual income, credit score, employment status, existing loan obligations, and number of dependents — to learn the underlying relationships that determine creditworthiness. Once trained, the model can generate a prediction in a fraction of a second, something that would take a human reviewer several minutes to hours to complete manually.

By automating the approval decision, the system substantially reduces the burden of manual verification on bank staff, allowing them to focus their expertise on edge cases and exceptions rather than routine applications. This not only increases throughput but also introduces a layer of objectivity into the decisioning process, since the model applies the same learned criteria uniformly across all applicants, reducing the risk of inconsistent or biased manual judgment. Furthermore, the system is built to deliver real-time predictions: once an applicant submits their details through the web interface, the trained model — persisted using Joblib as `credit_card_model.pkl` — is loaded into the Flask backend and used to generate an instantaneous Approved or Rejected outcome, along with supporting confidence information.

The solution is built as a complete end-to-end web application. The frontend, developed using HTML, CSS, and JavaScript, provides an intuitive form through which applicants can enter their details. The backend, built with Flask, handles request processing, input validation, data preprocessing using Pandas and NumPy, and communicates with the pre-trained Scikit-Learn model to generate predictions. For local development and testing, the application runs on a Flask development server, and for demonstration or sharing purposes, it can be exposed to the public internet using Ngrok, which creates a secure tunnel to the local server without requiring a dedicated cloud deployment.

Scenarios

- **Scenario 1 — High-Income Applicant with Excellent Credit Score:** An applicant with a high annual income, a strong credit score above 750, stable employment, and minimal existing debt submits an application through the web form. The system preprocesses the input, passes it through the trained model, and returns a prediction of Approved with a high confidence score, reflecting the applicant's strong financial profile.
- **Scenario 2 — Applicant with Low Credit Score:** An applicant with a low credit score, a history of missed payments, and a high ratio of existing loans to income submits their details. The model identifies the

elevated risk profile based on patterns learned during training and returns a prediction of Rejected, along with a lower confidence score indicating higher uncertainty or risk.

- **Scenario 3 — Applicant with Moderate Income and Average Credit History:** An applicant whose income and credit score fall in the moderate range, with a mixed history of existing loans and dependents, submits an application. In this borderline case the model weighs multiple contributing factors together and returns a prediction based on the combined analysis of all input features, demonstrating the nuanced decision boundary the model has learned rather than a simple threshold rule.

Technical Architecture

The Credit Card Approval Prediction System follows a modular, layered architecture that cleanly separates the presentation layer, the application logic layer, and the machine learning inference layer. This separation of concerns makes the system easier to maintain, test, and extend, while also allowing each layer to be developed and scaled independently.

Frontend Layer

- HTML — structures the applicant form and the prediction result page.
- CSS — provides a clean, responsive, and professional visual design for the application.
- JavaScript — handles client-side interactivity, asynchronous form submission using the Fetch API, and dynamic rendering of the prediction result.

Backend Layer

- Flask — a lightweight Python web framework that exposes HTTP routes, handles incoming requests, coordinates data preprocessing, and returns JSON or rendered HTML responses.

Machine Learning Layer

- Scikit-Learn — the pre-trained classification model that has learned the relationship between applicant attributes and the approval outcome.

Data Processing Layer

- Pandas — used to organize incoming applicant data into structured DataFrames that match the format expected by the model.
- NumPy — used for efficient numerical operations and array handling during preprocessing.

Model Storage

- Joblib — used to serialize and deserialize the trained model, allowing it to be saved once during training and loaded instantly at application startup as `credit_card_model.pkl`.

Deployment Layer

- Flask Local Server — runs the application on the developer's machine for local testing at `http://127.0.0.1:5000`.
- Ngrok Public Deployment — creates a secure tunnel that exposes the local Flask server to a publicly accessible URL, enabling remote access without cloud hosting.

Architecture Description: When a user submits the credit card application form, the JavaScript layer packages the form data into a JSON payload and sends it to the Flask backend via a POST request. The Flask route handler validates the incoming fields, converts them into a Pandas DataFrame with the same column structure used during training, and applies any necessary encoding or scaling. The processed data is then passed to the model that was loaded via Joblib at application startup. The model returns a predicted class (Approved or Rejected) along with a probability score, which the backend formats into a JSON response. The frontend receives this response and dynamically updates the page to display the outcome to the user, without requiring a full page reload.

Note: If the project is extended to persist applicant records or approval history in a database, an Entity-Relationship (ER) diagram describing the schema (e.g., Applicants, Applications, Predictions tables) should be added here along with a description of the relationships between entities.

Pre-Requisites

Before beginning development of the Credit Card Approval Prediction System, the following skills, tools, and frameworks should be understood:

- **Python Programming:** Core language used to implement the backend, data preprocessing, and model integration logic.
- **Flask Framework:** Used to build the web server, define routes, and handle HTTP requests and responses.
- **Machine Learning Basics:** Understanding of classification problems, training/testing splits, and evaluation metrics such as accuracy, precision, and recall.
- **Scikit-Learn:** Used for model training, evaluation, and inference within the application.
- **Pandas:** Used to structure and manipulate tabular applicant data.
- **NumPy:** Used for numerical computations and array-based operations during preprocessing.
- **HTML/CSS/JavaScript:** Used to build the responsive applicant-facing web interface.
- **Git & GitHub:** Used for source control, versioning, and collaborative development of the project.
- **Ngrok:** Used to expose the local Flask server to the internet for public demonstration and testing.

Project Workflow

The development of the Credit Card Approval Prediction System is organized into five major milestones, each broken down into specific activities. This milestone-based structure ensures that the project progresses systematically from data preparation through to public deployment.

Milestone 1: Data Collection and Model Development

- Activity 1.1: Dataset Collection
- Activity 1.2: Data Preprocessing
- Activity 1.3: Exploratory Data Analysis
- Activity 1.4: Model Training
- Activity 1.5: Model Evaluation

- Activity 1.6: Model Saving using Joblib

Milestone 2: Backend Development

- Activity 2.1: Flask Setup
- Activity 2.2: API Development
- Activity 2.3: Model Integration
- Activity 2.4: Input Validation

Milestone 3: App.py Development

- Activity 3.1: Home Route and Prediction Route Implementation

Milestone 4: Frontend Development

- Activity 4.1: Designing the User Interface
- Activity 4.2: Dynamic Rendering using JavaScript

Milestone 5: Deployment

- Activity 5.1: Local Deployment
- Activity 5.2: Local Testing
- Activity 5.3: Public Deployment using Ngrok

Milestone 1: Data Collection and Model Development

This milestone focuses on preparing a reliable dataset and building an accurate classification model capable of predicting whether a credit card application should be approved or rejected.

Activity 1.1: Dataset Collection

- Source a credit card approval dataset containing applicant attributes such as annual income, employment status, credit score, existing loans, number of dependents, and marital status, along with the historical approval outcome for each record.
- Ensure the dataset contains a sufficient number of both approved and rejected records to avoid class imbalance issues during training.

Activity 1.2: Data Preprocessing

- Handle missing values in numerical fields (such as income or credit score) using imputation strategies like mean or median substitution.
- Encode categorical fields such as Gender, Employment Status, and Marital Status using label encoding or one-hot encoding so they can be consumed by the model.
- Scale numerical features such as Annual Income and Loan Amount using standardization to bring all features onto a comparable range.
- Split the cleaned dataset into training and testing subsets, typically in an 80:20 ratio, to allow unbiased evaluation of the trained model.

Activity 1.3: Exploratory Data Analysis

- Analyse the distribution of the target variable (Approved vs Rejected) to understand class balance.
- Visualise relationships between key features (credit score, income) and the approval outcome using histograms, box plots, and correlation heatmaps.
- Identify and treat outliers in numerical fields such as income and loan amount that could otherwise skew the model.

Activity 1.4: Model Training

A classification algorithm from Scikit-Learn, such as Random Forest Classifier or Logistic Regression, is trained on the preprocessed training set. During training, the model learns the statistical relationship between the applicant's attributes and the historical approval outcome.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
model.fit(X_train, y_train)
```

Activity 1.5: Model Evaluation

- Evaluate the trained model on the held-out test set using metrics such as Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix.
- Compare multiple candidate algorithms if applicable, and select the model that offers the best balance between overall accuracy and the ability to correctly identify high-risk applicants (recall on the Rejected class).

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

Activity 1.6: Model Saving using Joblib

Once the final model has been selected and validated, it is serialized to disk using Joblib so that it can be loaded instantly by the Flask application without needing to be retrained every time the server starts.

```
import joblib

joblib.dump(model, 'credit_card_model.pkl')
print('Model saved successfully as credit_card_model.pkl')
```

Milestone 2: Backend Development

Milestone 2 focuses on building the Flask backend that will serve the web application, expose the prediction API, and integrate the trained machine learning model.

Activity 2.1: Flask Setup

- Install Flask and set up the base project structure, including folders for templates (HTML pages) and static assets (CSS and JavaScript files).

```
project/
├── static/
│   ├── css/
│   └── js/
├── templates/
│   └── index.html
├── app.py
├── credit_card_model.pkl
├── requirements.txt
└── run_public.py
```

Activity 2.2: API Development

- Define a home route (/) that renders the applicant form.
- Define a prediction route (/predict) that accepts POST requests containing applicant data and returns a JSON response with the predicted outcome.

Activity 2.3: Model Integration

- Load the serialized `credit_card_model.pkl` file once at application startup using Joblib, so the model remains in memory and ready for fast inference on every incoming request.

```
import joblib

model = joblib.load('credit_card_model.pkl')
```

Activity 2.4: Input Validation

- Verify that all required fields (Applicant Name, Age, Gender, Annual Income, Employment Status, Credit Score, Loan Amount, Number of Dependents, Existing Loans, Marital Status) are present in the incoming request.
- Ensure numeric fields such as Age, Annual Income, Credit Score, and Loan Amount fall within sensible, expected ranges before being passed to the model.
- Return a descriptive 400 error response for any missing or malformed fields so that the frontend can display an appropriate message to the applicant.

Milestone 3: App.py Development

Milestone 3 focuses on writing the core application logic in `app.py`, which acts as the backbone of the Credit Card Approval Prediction System. This file wires together routing, input validation, preprocessing, and the model inference call.

Home Route

The home route renders the main HTML interface where the applicant fills in their details.

```
@app.route("/")
def home():
    return render_template("index.html")
```

Prediction Route

The prediction route is a POST endpoint that accepts the applicant's form data as JSON, validates and preprocesses it, runs it through the trained model, and returns a structured JSON response containing the prediction and a confidence score.

```
@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json()

    required_fields = [
        "age", "gender", "annual_income", "employment_status",
        "credit_score", "loan_amount", "dependents",
        "existing_loans", "marital_status"
    ]

    missing = [f for f in required_fields if f not in data or data[f] in ("", None)]
    if missing:
```

```

        return jsonify({"error": f"Missing required fields: {'', '.join(missing)}"}), 400

    try:
        input_df = preprocess_input(data)
        prediction = model.predict(input_df)[0]
        probability = model.predict_proba(input_df)[0].max()

        result = "Approved" if prediction == 1 else "Rejected"

        return jsonify({
            "success": True,
            "prediction": result,
            "confidence": round(float(probability) * 100, 2)
        })
    except Exception as e:
        return jsonify({"error": f"Prediction failed: {str(e)}"}), 500

```

Request Handling and Data Validation

Every incoming request is read using Flask's `request.get_json()` method. The backend first checks that every required field is present and non-empty before proceeding, returning a 400 status code with a descriptive error message if any field is missing.

Preprocessing

The `preprocess_input()` helper function converts the raw JSON payload into a Pandas DataFrame with the exact column names and encodings used during model training, applying the same label encoders and scalers that were fitted during Milestone 1.

```

def preprocess_input(data):
    df = pd.DataFrame([
        "age": int(data["age"]),
        "gender": gender_encoder.transform([data["gender"]])[0],
        "annual_income": float(data["annual_income"]),
        "employment_status": employment_encoder.transform([data["employment_status"]])[0],
        "credit_score": int(data["credit_score"]),
        "loan_amount": float(data["loan_amount"]),
        "dependents": int(data["dependents"]),
        "existing_loans": int(data["existing_loans"]),
        "marital_status": marital_encoder.transform([data["marital_status"]])[0],
    ])
    df[numeric_cols] = scaler.transform(df[numeric_cols])
    return df

```

Prediction Logic

The preprocessed DataFrame is passed to `model.predict()` to obtain the binary outcome (1 for Approved, 0 for Rejected) and to `model.predict_proba()` to obtain the associated confidence score, which reflects how strongly the model favours the predicted class.

Error Handling

All prediction logic is wrapped in a try/except block. Validation errors are caught explicitly and returned as 400 responses, while any unexpected exception during preprocessing or inference is caught by a general exception

handler and returned as a 500 response with a descriptive message, ensuring the frontend always receives a meaningful JSON error rather than a server crash.

JSON Response

On success, the endpoint returns a JSON object containing a success flag, the predicted outcome (Approved or Rejected), and the confidence percentage, which the frontend uses to render the result to the applicant.

Milestone 4: Frontend Development

Milestone 4 focuses on building a clean, professional, and responsive user interface that allows applicants to submit their details and view the prediction result without needing a full page reload.

Activity 4.1: Designing the User Interface

The applicant form is built as a single, well-organised HTML page containing input fields for all attributes required by the model:

- Applicant Name
- Age
- Gender
- Annual Income
- Employment Status
- Credit Score
- Loan Amount
- Number of Dependents
- Existing Loans
- Marital Status

Each field is paired with appropriate input types (text, number, and dropdown/select elements for categorical fields such as Gender, Employment Status, and Marital Status) and client-side constraints to reduce invalid submissions before the request even reaches the backend. The form is styled using CSS to present a clean, card-based layout that is easy to read and fill out on both desktop and mobile devices.

Activity 4.2: Dynamic Rendering using JavaScript

- **Fetch API:** The form's submit event is intercepted using JavaScript, which prevents the default page reload and instead sends the collected form data as a JSON payload to the /predict endpoint using the Fetch API.
- **Loading Spinner:** While the request is in flight, the submit button is disabled and replaced with an animated loading spinner to give the applicant clear feedback that their request is being processed.
- **Prediction Display:** Once the response is received, JavaScript dynamically updates the result section of the page, displaying the prediction (Approved or Rejected) along with the associated confidence score in a clearly styled result card.
- **Error Messages:** If the backend returns an error (for example, due to missing or invalid fields), JavaScript displays a clear, user-friendly error message inline on the form rather than failing silently.

```

form.addEventListener("submit", async (e) => {
  e.preventDefault();
  submitBtn.disabled = true;
  showLoadingSpinner();

  try {
    const response = await fetch("/predict", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(collectFormData())
    });
    const data = await response.json();

    if (!response.ok) {
      showError(data.error);
    } else {
      renderResult(data.prediction, data.confidence);
    }
  } catch (err) {
    showError("Something went wrong. Please try again.");
  } finally {
    submitBtn.disabled = false;
    hideLoadingSpinner();
  }
});

```

Milestone 5: Deployment

Milestone 5 covers deploying the Credit Card Approval Prediction System, first locally to verify correctness, and then publicly using Ngrok so that the application can be accessed and demonstrated from anywhere.

Activity 5.1: Local Deployment

- Creating a Virtual Environment:

```
python -m venv myenv
```

- Activating the Environment (Windows):

```
myenv\Scripts\activate
```

- Installing Requirements:

```
pip install -r requirements.txt
```

- Running the Flask Application:

```
python app.py
```

- Opening the application in a browser:

```
http://127.0.0.1:5000
```

Activity 5.2: Local Testing

- Testing Valid Inputs: Submit applications with realistic, well-formed data across a range of income, credit score, and loan amount values to confirm the model returns sensible Approved/Rejected predictions.
- Testing Invalid Inputs: Submit forms with missing fields, negative numbers, or out-of-range values to confirm the backend correctly rejects them with a descriptive error message.
- Verifying Predictions: Cross-check a sample of predictions against the expected outcome based on the applicant's profile to build confidence in the model's behaviour.
- Checking Frontend-Backend Integration: Confirm that the loading spinner, result rendering, and error messages all behave correctly end-to-end through the browser.

Activity 5.3: Public Deployment using Ngrok

- Installing pyngrok:

```
pip install pyngrok
```

- Installing the Ngrok client from <https://ngrok.com/download> and configuring the authentication token obtained from the Ngrok dashboard:

```
ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Running the public deployment script, which starts both the Ngrok tunnel and the Flask server together:

```
python run_public.py
```

- Sample public URL printed to the terminal once the tunnel is established:

```
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```

Important: debug=False is recommended in run_public.py to prevent Flask from spawning a reloader process, which can interfere with Ngrok's tunnel management. On the free Ngrok plan, the public URL changes each time the script is restarted, and free-tier sessions may expire after a period of inactivity — simply restart run_public.py to obtain a new URL.

Exploring the Website's Web Pages

Home Page

The Home Page serves as the single-page interface for the application, combining the landing view and the prediction form. It presents a clean header identifying the Credit Card Approval Prediction System, followed by the applicant input form. The layout is responsive, adapting cleanly across desktop, tablet, and mobile screen sizes, and uses clear visual grouping to separate personal details, financial details, and existing obligations.

Applicant Form

The Applicant Form section contains labeled input fields for Applicant Name, Age, Gender, Annual Income, Employment Status, Credit Score, Loan Amount, Number of Dependents, Existing Loans, and Marital Status. Dropdown selectors are used for categorical fields to prevent free-text errors, and numeric fields include basic client-side constraints (such as minimum and maximum values) to guide the applicant toward valid input before submission. A prominent "Predict" button submits the form asynchronously to the backend.

Prediction Result Section

Once a prediction is returned, the Prediction Result section is dynamically revealed below the form without a page reload. It clearly displays the outcome — Approved or Rejected — using distinct colour cues (for example, green for Approved and red for Rejected), along with the model's confidence score expressed as a percentage. A short feedback message accompanies the result, and the applicant is given the option to submit a new application.

Technologies Used

Category	Technologies
Frontend	HTML5, CSS3, JavaScript
Backend	Python, Flask
Machine Learning	Scikit-Learn
Data Processing	Pandas, NumPy
Model Storage	Joblib
Deployment	Ngrok
Version Control	Git & GitHub

Expected Output

The expected end-to-end behaviour of the Credit Card Approval Prediction System is as follows:

- The applicant navigates to the Home Page and enters their personal, financial, and employment details into the form.
- Upon clicking the Predict button, the form data is submitted to the backend.
- The backend validates all input fields, ensuring required values are present and within acceptable ranges.
- The validated data is preprocessed and passed to the trained model to run the prediction.
- The system displays one of two possible outcomes:
 - Approved — the applicant meets the learned criteria for creditworthiness.
 - Rejected — the applicant's profile falls below the learned approval threshold.
- Alongside the outcome, a confidence score is displayed, indicating how strongly the model supports its prediction.
- Contextual feedback is shown to the applicant, helping them understand the general basis of the decision.

Conclusion

The Credit Card Approval Prediction System Using Machine Learning successfully demonstrates how a trained Scikit-Learn classification model can be integrated into a complete, production-style Flask web application to automate a traditionally manual banking process. By combining Pandas and NumPy for data preprocessing, Joblib for efficient model persistence, and a responsive HTML, CSS, and JavaScript frontend, the project delivers real-time, consistent, and objective predictions of Approved or Rejected outcomes for credit card applicants. The application was validated through structured local testing across a range of input scenarios and made accessible for demonstration through Ngrok's secure public tunneling, without requiring a dedicated cloud deployment.

From a banking automation perspective, the project highlights the tangible benefits that machine learning brings to credit decisioning: dramatically reduced turnaround time compared to manual review, consistent application of learned decision criteria across all applicants, and the ability to scale to high volumes of applications without a proportional increase in staffing. These benefits position systems of this kind as a valuable complement to, rather than a wholesale replacement for, experienced human underwriters, who remain essential for reviewing borderline or exceptional cases.

Future Scope

- **Deep Learning Models:** Exploring neural network based architectures to potentially capture more complex, non-linear relationships in applicant data.
- **Real-time Bank Integration:** Connecting the system directly to core banking systems and credit bureau APIs for live data ingestion rather than manually entered form data.
- **Cloud Deployment:** Migrating the application from local Flask/Ngrok hosting to a scalable cloud platform for production-grade availability and reliability.
- **Explainable AI:** Incorporating model interpretability techniques such as SHAP or LIME to provide transparent, applicant-facing explanations for approval or rejection decisions.

- **User Authentication:** Adding secure login for bank staff and applicants to control access to the system and protect sensitive financial data.
- **Applicant History Tracking:** Persisting past applications and predictions in a database to support auditing, trend analysis, and longitudinal risk assessment.